

Lesson 8 Notes: Lower Triangular Matrix in C++

1. Main idea of Lesson 8

Lesson 8 teaches how to implement a **lower triangular matrix in C++**.

In the previous lesson, Lesson 7, we implemented a lower triangular matrix in **C** using:

- `struct Matrix`
- `malloc()`
- `free()`
- separate functions such as `set`, `get`, and `display`

In Lesson 8, we use **C++ object-oriented programming**.

That means we use a **class** to keep the matrix data and matrix operations together.

The matrix type does not change.

The rule is still:

```
i >= j
```

This means:

```
If i >= j, the position is stored.  
If i < j, the position is treated as zero.
```

So Lesson 8 is mainly about converting the lower triangular matrix program from C style into C++ class style.

2. What you should already know before Lesson 8

Before Lesson 8, you should understand these ideas:

Concept	Meaning
Special matrix	A matrix with a useful pattern that helps save memory.
Structural zero	A zero that exists because of the matrix rule, not because it is stored.

Concept	Meaning
1D array storage	Storing only useful matrix values in one array.
Diagonal matrix	Stores only positions where <code>i == j</code> .
Lower triangular matrix	Stores positions where <code>i >= j</code> .
Row-major storage	Stores values row by row.
Column-major storage	Stores values column by column.
C++ class	A blueprint that groups data and functions together.
Constructor	A special function that runs when an object is created.
Destructor	A special function that runs when an object is destroyed.
Dynamic memory	Memory created during program execution using <code>new</code> .
<code>delete[]</code>	Used to release dynamic arrays created using <code>new[]</code> .

3. Current progress review

So far, the course has moved like this:

Lessons	What you learned	Main idea
Lessons 1–4	Diagonal matrix	Store only values where <code>i == j</code> .
Lesson 5	Lower triangular row-major mapping	Store values where <code>i >= j</code> , row by row.
Lesson 6	Lower triangular column-major mapping	Same matrix shape, but stored column by column.
Lesson 7	Lower triangular matrix in C	Use <code>struct Matrix</code> , <code>malloc()</code> , <code>free()</code> , <code>set</code> , <code>get</code> , and <code>display</code> .
Lesson 8	Lower triangular matrix in C++	Use a C++ class with constructors, destructor, and member functions.

The biggest jump in Lesson 8 is this:

```

C struct + separate functions
    ↓
C++ class + member functions

```

The matrix logic stays the same.

The implementation style changes.

4. What is a lower triangular matrix?

A **lower triangular matrix** is a square matrix where the useful values are on or below the main diagonal.

Example:

```
1 0 0 0
2 3 0 0
4 5 6 0
7 8 9 10
```

The useful part is:

```
1
2 3
4 5 6
7 8 9 10
```

This useful part forms a triangle in the lower-left part of the matrix.

That is why it is called a **lower triangular matrix**.

5. Stored condition

For a lower triangular matrix, a position is stored when:

```
i >= j
```

Where:

```
i = row number
j = column number
```

Examples:

Position	Check	Stored?
M[1,1]	1 >= 1	Yes
M[2,1]	2 >= 1	Yes
M[2,2]	2 >= 2	Yes
M[3,1]	3 >= 1	Yes
M[4,3]	4 >= 3	Yes
M[4,4]	4 >= 4	Yes

So the main condition is:

```
if (i >= j)
```

If this condition is true, the value belongs to the lower triangle.

6. Zero condition

For a lower triangular matrix, a position is zero when:

```
i < j
```

This means the position is above the main diagonal.

Examples:

Position	Check	Meaning
M[1,2]	1 < 2	Structural zero
M[1,3]	1 < 3	Structural zero
M[2,4]	2 < 4	Structural zero
M[3,4]	3 < 4	Structural zero

These values are not stored in the array.

The program simply returns or prints 0 when needed.

7. Structural zeros

A **structural zero** is a zero that exists because of the matrix structure.

In a lower triangular matrix, every value above the main diagonal is a structural zero.

Example:

$$M[2,4] = 0$$

Why?

Because:

$$2 < 4$$

So $(2,4)$ is above the main diagonal.

That position is not stored.

The program already knows it must be zero.

8. Stored region vs zero region

For this matrix:

```
1 0 0 0
2 3 0 0
4 5 6 0
7 8 9 10
```

Stored positions are:

```
M[1,1]
M[2,1] M[2,2]
M[3,1] M[3,2] M[3,3]
M[4,1] M[4,2] M[4,3] M[4,4]
```

Zero positions are:

```
M[1,2] M[1,3] M[1,4]
M[2,3] M[2,4]
M[3,4]
```

Stored condition:

```
i >= j
```

Zero condition:

```
i < j
```

9. Why lower triangular storage saves memory

A normal $N \times N$ matrix stores:

```
N2 values
```

A lower triangular matrix stores only:

```
N(N + 1) / 2 values
```

This is about half of the full matrix.

Example for $N = 4$:

```
Full matrix values = 4 × 4 = 16
Lower triangular stored values = 4(4 + 1) / 2
                                = 4(5) / 2
                                = 20 / 2
                                = 10
```

So for a 4×4 lower triangular matrix, we store only 10 values instead of 16.

10. Number of structural zeros

The number of structural zeros above the main diagonal is:

$$N(N - 1) / 2$$

For $N = 4$:

$$\begin{aligned}\text{Zeros} &= 4(4 - 1) / 2 \\ &= 4(3) / 2 \\ &= 12 / 2 \\ &= 6\end{aligned}$$

So a 4×4 lower triangular matrix has:

10 stored values
6 structural zeros
16 total visible positions

11. Why Lesson 8 uses a C++ class

In C, Lesson 7 used this structure:

```
struct Matrix {  
    int *A;  
    int n;  
};
```

In C++, Lesson 8 uses a class:

```
class LowerTri {  
private:  
    int *A;  
    int n;  
};
```

A class is useful because it keeps related data and related functions together.

For a lower triangular matrix, the data are:

```
int *A;  
int n;
```

The functions are:

```
Set()  
Get()  
Display()
```

So the object manages its own matrix.

12. Class vs struct idea

In Lesson 7, we had:

```
Data stored in a struct.  
Functions written separately.
```

In Lesson 8, we have:

```
Data and functions inside one class.
```

Example C style from Lesson 7:

```
set(&m, i, j, x);  
display(m);
```

Example C++ style from Lesson 8:

```
lm.Set(i, j, x);  
lm.Display();
```

Here, `lm` is an object of the `LowerTri` class.

The object calls its own functions.

13. Basic class design

The class structure is:

```
class LowerTri {  
private:  
    int *A;  
    int n;  
  
public:  
    LowerTri();  
    LowerTri(int n);  
    ~LowerTri();  
    void Set(int i, int j, int x);  
    int Get(int i, int j);  
    void Display();  
};
```

The class has two main sections:

```
private:
```

and:

```
public:
```

14. Private section

The private section is:

```
private:  
    int *A;  
    int n;
```

These values should not be directly accessed from outside the class.

Meaning of `int *A`

```
int *A;
```

This means `A` is a pointer to an integer array.

This dynamic array stores only the useful lower triangular values.

Example:

```
A = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

This array represents the full matrix:

```
1 0 0 0
2 3 0 0
4 5 6 0
7 8 9 10
```

Meaning of `int n`

```
int n;
```

This stores the dimension of the matrix.

If:

```
n = 4;
```

then the object represents a:

```
4 × 4 matrix
```

15. Why `A` and `n` are private

The data members are private so outside code cannot directly change them incorrectly.

For example, outside code should not do this:

```
lm.A[3] = 99;
lm.n = 100;
```

That would be unsafe.

Instead, outside code should use:

```
lm.Set(i, j, x);
```

The `Set()` method checks the lower triangular rule before storing the value.

So private data helps protect the matrix structure.

16. Public section

The public section contains functions that outside code is allowed to call.

```
public:  
    LowerTri();  
    LowerTri(int n);  
    ~LowerTri();  
    void Set(int i, int j, int x);  
    int Get(int i, int j);  
    void Display();
```

Public function	Purpose
<code>LowerTri()</code>	Creates a default matrix.
<code>LowerTri(int n)</code>	Creates a matrix with a user-given dimension.
<code>~LowerTri()</code>	Releases dynamic memory.
<code>Set()</code>	Stores a value if it belongs to the lower triangle.
<code>Get()</code>	Returns a stored value or returns <code>0</code> .
<code>Display()</code>	Prints the full visible matrix.

17. Default constructor

A default constructor has no parameters.

Example:

```
LowerTri() {  
    n = 2;  
    A = new int[3];  
}
```

This creates a default `2 × 2` lower triangular matrix.

Why `3` cells?

Because:

```
n(n + 1) / 2  
= 2(2 + 1) / 2  
= 2(3) / 2  
= 3
```

So a `2 × 2` lower triangular matrix stores `3` values.

The visible matrix shape is:

```
? 0  
? ?
```

The three stored positions are:

```
M[1,1]  
M[2,1]  
M[2,2]
```

18. Safer default constructor

A safer beginner-friendly constructor is:

```
LowerTri() {  
    n = 2;  
    A = new int[n * (n + 1) / 2]{};  
}
```

The `{}` initializes all array values to zero.

Without `{}`, the array may contain garbage values before we store real values.

So this:

```
A = new int[n * (n + 1) / 2]{};
```

is safer than this:

```
A = new int[n * (n + 1) / 2];
```

19. Parameterized constructor

The parameterized constructor receives the matrix size.

```
LowerTri(int n) {  
    this->n = n;  
    A = new int[n * (n + 1) / 2]{};  
}
```

If we write:

```
LowerTri lm(4);
```

then the constructor receives:

```
n = 4
```

The array size becomes:

```
4(4 + 1) / 2 = 10
```

So the object can represent a `4 × 4` lower triangular matrix using only `10` stored values.

20. Understanding `this->n = n`

The constructor has this line:

```
this->n = n;
```

There are two different `n` names here.

The first one is:

```
this->n
```

This means the object's own private data member `n`.

The second one is:

```
n
```

This means the constructor parameter.

So:

```
this->n = n;
```

means:

Store the parameter value into the object's own `n` variable.

Example:

```
LowerTri lm(4);
```

The parameter `n` is `4`, so the object's `n` becomes `4`.

21. Dynamic memory using `new`

In C++, dynamic arrays are created using:

```
new int[size]
```

For a lower triangular matrix, the number of stored values is:

```
n * (n + 1) / 2
```

So we allocate:

```
A = new int[n * (n + 1) / 2]{};
```

We do not allocate:

```
A = new int[n * n];
```

because that would store the full matrix and waste memory.

We also do not allocate:

```
A = new int[n];
```

because `n` cells are enough only for a diagonal matrix, not for a lower triangular matrix.

22. Destructor

A destructor runs automatically when the object is destroyed.

The destructor is:

```
~LowerTri() {  
    delete[] A;  
}
```

It releases the dynamic array.

Since the constructor uses:

```
new int[...]
```

we must use:

```
delete[] A;
```

23. Important memory rule

The important memory rule is:

```
new[] must be matched with delete[]
```

Correct:

```
A = new int[n * (n + 1) / 2]{};
delete[] A;
```

Wrong:

```
A = new int[n * (n + 1) / 2]{};
delete A;
```

Why is `delete A` wrong?

Because `A` points to an array, not one single integer.

For arrays, use:

```
delete[] A;
```

24. Row-major storage reminder

Lesson 8 first implements the matrix using **row-major storage**.

Row-major means:

```
Store row 1.
Then row 2.
```


Then row 3.
Then row 4.

Example matrix:

```
1 0 0 0
2 3 0 0
4 5 6 0
7 8 9 10
```

Row-major storage:

```
Row 1: 1
Row 2: 2 3
Row 3: 4 5 6
Row 4: 7 8 9 10
```

So the array becomes:

```
A = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

25. Row-major mapping table

For this matrix:

```
1 0 0 0
2 3 0 0
4 5 6 0
7 8 9 10
```

The row-major mapping is:

Matrix position	Array position
M[1,1]	A[0]
M[2,1]	A[1]
M[2,2]	A[2]
M[3,1]	A[3]

Matrix position	Array position
M[3,2]	A[4]
M[3,3]	A[5]
M[4,1]	A[6]
M[4,2]	A[7]
M[4,3]	A[8]
M[4,4]	A[9]

26. Row-major index formula

For lower triangular row-major storage, the formula is:

$$\text{index} = i * (i - 1) / 2 + (j - 1)$$

Use this formula only when:

$$i \geq j$$

If:

$$i < j$$

then the position is above the diagonal and is not stored.

27. Meaning of the row-major formula

The formula is:

$$\text{index} = i * (i - 1) / 2 + (j - 1)$$

It has two parts.

Part 1: Skip previous rows

$$i * (i - 1) / 2$$

This counts how many stored values come before row `i`.

Example:

If `i = 4`, the previous rows are:

Row 1 stores 1 value.
Row 2 stores 2 values.
Row 3 stores 3 values.

Total values before row 4:

$$1 + 2 + 3 = 6$$

Using the formula:

$$4(4 - 1) / 2 = 4(3) / 2 = 6$$

Part 2: Move inside the current row

$$j - 1$$

This moves to the correct column inside the current row.

Examples:

Column	Movement
<code>j = 1</code>	0 steps
<code>j = 2</code>	1 step
<code>j = 3</code>	2 steps
<code>j = 4</code>	3 steps

So the formula means:

index = number of stored values before this row + movement inside this row

28. Dry run: index of `M[4,3]`

We want:

`M[4,3]`

So:

`i = 4`
`j = 3`

Step 1: Check condition.

`4 >= 3`

This is true.

So the position is stored.

Step 2: Use the row-major formula.

`index = i * (i - 1) / 2 + (j - 1)`

Substitute values:

`index = 4 * (4 - 1) / 2 + (3 - 1)`
`index = 4 * 3 / 2 + 2`
`index = 12 / 2 + 2`
`index = 6 + 2`
`index = 8`

So:

`M[4,3] -> A[8]`

29. Dry run: index of `M[2,4]`

We want:

`M[2,4]`

So:

`i = 2`
`j = 4`

Step 1: Check condition.

`2 >= 4`

This is false.

So the position is not stored.

Since:

`2 < 4`

`M[2,4]` is a structural zero.

So:

`M[2,4] = 0`

No array index should be calculated for it.

30. Set method

The `Set()` method stores a value in the matrix.

```
void Set(int i, int j, int x) {  
    if (i >= j) {
```

```
        int index = (i * (i - 1) / 2) + (j - 1);  
        A[index] = x;  
    }  
}
```

Meaning:

```
i = row number  
j = column number  
x = value to store
```

The method checks:

```
if (i >= j)
```

If true, the position belongs to the lower triangle.

Then it calculates the index and stores `x`.

If false, the method does nothing.

Why?

Because positions above the diagonal must stay zero.

31. Dry run of `Set()` for a stored position

Call:

```
lm.Set(4, 3, 9);
```

Values:

```
i = 4  
j = 3  
x = 9
```

Step 1: Check condition.

```
4 >= 3
```

This is true.

Step 2: Calculate index.

```
index = 4 * (4 - 1) / 2 + (3 - 1)
index = 4 * 3 / 2 + 2
index = 6 + 2
index = 8
```

Step 3: Store value.

```
A[8] = 9;
```

So `9` is stored at `A[8]`.

32. Dry run of `Set()` for a structural zero

Call:

```
lm.Set(2, 4, 99);
```

Values:

```
i = 2
j = 4
x = 99
```

Step 1: Check condition.

```
2 >= 4
```

This is false.

So `(2,4)` is above the main diagonal.

Step 2: Do nothing.

The value `99` is ignored.

Why?

Because `M[2,4]` must be zero in a lower triangular matrix.

33. Get method

The `Get()` method returns the value at a matrix position.

```
int Get(int i, int j) {  
    if (i >= j) {  
        int index = (i * (i - 1) / 2) + (j - 1);  
        return A[index];  
    }  
    return 0;  
}
```

Meaning:

If $i \geq j$, calculate the index and return `A[index]`.
If $i < j$, return 0.

The method returns `0` for positions above the diagonal because they are structural zeros.

34. Dry run of `Get()` for a stored position

Suppose the array represents this matrix:

```
1 0 0 0  
2 3 0 0  
4 5 6 0  
7 8 9 10
```

Stored array:

```
A = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

Call:


```
lm.Get(4, 3);
```

Values:

```
i = 4  
j = 3
```

Step 1: Check condition.

```
4 >= 3
```

This is true.

Step 2: Calculate index.

```
index = 4 * (4 - 1) / 2 + (3 - 1)  
index = 4 * 3 / 2 + 2  
index = 6 + 2  
index = 8
```

Step 3: Return value.

```
return A[8];
```

Since:

```
A[8] = 9
```

The answer is:

```
9
```

35. Dry run of `Get()` for a structural zero

Call:

```
lm.Get(2, 4);
```

Values:

```
i = 2  
j = 4
```

Step 1: Check condition.

```
2 >= 4
```

This is false.

So (2,4) is above the main diagonal.

Step 2: Return zero.

```
return 0;
```

The method does not read the array.

It already knows the value must be zero.

36. Why Get() returns 0, not -1

A helper index function may return -1 to mean:

```
This position is not stored.
```

But Get() should return the actual matrix value.

For a lower triangular matrix, positions above the diagonal are actual zeros.

So Get() should return:

```
return 0;
```

not:

```
return -1;
```

37. Display method

The `Display()` method prints the full visible matrix.

```
void Display() {  
    for (int i = 1; i <= n; i++) {  
        for (int j = 1; j <= n; j++) {  
            if (i >= j) {  
                int index = (i * (i - 1) / 2) + (j - 1);  
                cout << A[index] << " ";  
            } else {  
                cout << "0 ";  
            }  
        }  
        cout << endl;  
    }  
}
```

The stored array contains only useful values.

But the user should see the full square matrix.

So `Display()` uses nested loops.

For every position:

If $i \geq j$, print the stored value.
If $i < j$, print 0.

38. Why `Display()` uses nested loops

A 4×4 matrix has:

$4 \times 4 = 16$ visible positions

Even though we store only 10 values, the display must print all 16 positions.

So we need:

```
for (int i = 1; i <= n; i++) {  
    for (int j = 1; j <= n; j++) {  
        // print each position  
    }  
}
```

The outer loop controls rows.

The inner loop controls columns.

39. Dry run of `Display()`

Assume:

```
n = 4  
A = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

The full matrix should be:

```
1 0 0 0  
2 3 0 0  
4 5 6 0  
7 8 9 10
```

Row 1

Positions:

```
(1,1), (1,2), (1,3), (1,4)
```

- (1,1): `1 >= 1`, print `A[0] = 1`
- (1,2): `1 < 2`, print `0`
- (1,3): `1 < 3`, print `0`
- (1,4): `1 < 4`, print `0`

Output:

1 0 0 0

Row 2

- (2,1) : 2 >= 1 , print A[1] = 2
- (2,2) : 2 >= 2 , print A[2] = 3
- (2,3) : 2 < 3 , print 0
- (2,4) : 2 < 4 , print 0

Output:

2 3 0 0

Row 3

- (3,1) : print A[3] = 4
- (3,2) : print A[4] = 5
- (3,3) : print A[5] = 6
- (3,4) : print 0

Output:

4 5 6 0

Row 4

- (4,1) : print A[6] = 7
- (4,2) : print A[7] = 8
- (4,3) : print A[8] = 9
- (4,4) : print A[9] = 10

Output:

7 8 9 10

Full output:

1 0 0 0
2 3 0 0
4 5 6 0
7 8 9 10

40. Full input reading strategy

The program asks the user to enter all `N × N` matrix values.

Example for `n = 4`:

```
1 0 0 0
2 3 0 0
4 5 6 0
7 8 9 10
```

The program reads every value:

```
for (int i = 1; i <= d; i++) {
    for (int j = 1; j <= d; j++) {
        cin >> x;
        lm.Set(i, j, x);
    }
}
```

This means `Set()` is called for every matrix position.

But `Set()` only stores values where:

```
i >= j
```

So values above the diagonal are ignored automatically.

41. Why reading full input is useful

The user can type the matrix normally as a full square matrix.

Example:

```
1 0 0 0
2 3 0 0
4 5 6 0
7 8 9 10
```

This is easy for the user.

The user does not need to know the internal 1D array storage order.

The class handles the storage internally.

This is one of the advantages of using methods such as `Set()`.

42. Full C++ program: row-major lower triangular matrix

```
#include <iostream>
using namespace std;

class LowerTri {
private:
    int *A;
    int n;

public:
    LowerTri() {
        n = 2;
        A = new int[n * (n + 1) / 2]{};
    }

    LowerTri(int n) {
        this->n = n;
        A = new int[n * (n + 1) / 2]{};
    }

    ~LowerTri() {
        delete[] A;
    }

    void Set(int i, int j, int x) {
        if (i >= j) {
            int index = (i * (i - 1) / 2) + (j - 1);
            A[index] = x;
        }
    }

    int Get(int i, int j) {
        if (i >= j) {
            int index = (i * (i - 1) / 2) + (j - 1);
            return A[index];
        }
    }
}
```

```

        return 0;
    }

    void Display() {
        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= n; j++) {
                if (i >= j) {
                    int index = (i * (i - 1) / 2) + (j - 1);
                    cout << A[index] << " ";
                } else {
                    cout << "0 ";
                }
            }
            cout << endl;
        }
    };

    int main() {
        int d;
        int x;

        cout << "Enter dimension: ";
        cin >> d;

        LowerTri lm(d);

        cout << "Enter all matrix elements:" << endl;

        for (int i = 1; i <= d; i++) {
            for (int j = 1; j <= d; j++) {
                cin >> x;
                lm.Set(i, j, x);
            }
        }

        cout << endl << "Matrix is:" << endl;
        lm.Display();

        return 0;
    }

```


43. Explanation of the full program

Header file

```
#include <iostream>
```

This is needed because the program uses:

```
cin  
cout  
endl
```

Namespace

```
using namespace std;
```

This allows us to write:

```
cout  
cin  
endl
```

instead of:

```
std::cout  
std::cin  
std::endl
```

Class name

```
class LowerTri
```

This creates a class named `LowerTri`.

The class represents a lower triangular matrix.

Private data

```
int *A;  
int n;
```

This stores:

```
A = dynamic array of lower triangular values  
n = matrix dimension
```

Constructors

```
LowerTri()  
LowerTri(int n)
```

These create valid matrix objects.

Destructor

```
~LowerTri()
```

This releases the dynamic memory.

Set method

```
void Set(int i, int j, int x)
```

This stores `x` only if `i >= j`.

Get method

```
int Get(int i, int j)
```

This returns:

```
stored value if i >= j  
0 if i < j
```

Display method

```
void Display()
```

This prints the full visible matrix.

Main function

The main function:

1. Reads the dimension.
2. Creates a `LowerTri` object.
3. Reads all matrix values.
4. Calls `Set()` for every input.
5. Displays the matrix.
6. Ends the program.

44. Full input dry run for `n = 4`

Suppose:

```
dimension = 4
```

The storage size is:

```
4(4 + 1) / 2 = 10
```

So the array has positions:

```
A[0] to A[9]
```

Suppose the user enters:

```
1 0 0 0
2 3 0 0
4 5 6 0
7 8 9 10
```

Row 1 input

1 0 0 0

Calls:

```
Set(1,1,1) -> stored  
Set(1,2,0) -> ignored  
Set(1,3,0) -> ignored  
Set(1,4,0) -> ignored
```

Only **1** is stored.

Row 2 input

2 3 0 0

Calls:

```
Set(2,1,2) -> stored  
Set(2,2,3) -> stored  
Set(2,3,0) -> ignored  
Set(2,4,0) -> ignored
```

Only **2** and **3** are stored.

Row 3 input

4 5 6 0

Calls:

```
Set(3,1,4) -> stored  
Set(3,2,5) -> stored  
Set(3,3,6) -> stored  
Set(3,4,0) -> ignored
```

Only **4**, **5**, and **6** are stored.

Row 4 input

```
7 8 9 10
```

Calls:

```
Set(4,1,7) -> stored  
Set(4,2,8) -> stored  
Set(4,3,9) -> stored  
Set(4,4,10) -> stored
```

All four values are stored.

Final stored array:

```
A = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

Displayed matrix:

```
1 0 0 0  
2 3 0 0  
4 5 6 0  
7 8 9 10
```

45. Column-major formula switch

Lesson 8 also explains how to switch the class from row-major storage to column-major storage.

The matrix type does not change.

It is still lower triangular.

The stored condition does not change:

```
i >= j
```

The zero condition does not change:

$$i < j$$

The array size does not change:

$$n * (n + 1) / 2$$

Only the index formula changes.

46. Column-major formula

For lower triangular column-major storage, the formula is:

$$\text{index} = n * (j - 1) - ((j - 2) * (j - 1) / 2) + (i - j)$$

Use this formula only when:

$$i \geq j$$

The formula means:

$$\text{index} = \text{skipped columns} + \text{movement inside the current column}$$

47. Row-major vs column-major formula comparison

Storage type	Formula
Row-major lower triangular	$\text{index} = i * (i - 1) / 2 + (j - 1)$
Column-major lower triangular	$\text{index} = n * (j - 1) - ((j - 2) * (j - 1) / 2) + (i - j)$

The condition stays the same:

$$i \geq j$$

The only thing that changes is the index formula.

48. How to convert the class to column-major

To convert from row-major to column-major, replace this formula in `Set()`, `Get()`, and `Display()`:

```
int index = (i * (i - 1) / 2) + (j - 1);
```

with this formula:

```
int index = (n * (j - 1)) - ((j - 2) * (j - 1) / 2) + (i - j);
```

Do not change:

```
if (i >= j)
```

Do not change:

```
n * (n + 1) / 2
```

Do not change the main program.

Only change the formula lines.

49. Column-major `Set()` version

```
void Set(int i, int j, int x) {  
    if (i >= j) {  
        int index = (n * (j - 1)) - ((j - 2) * (j - 1) / 2) + (i - j);  
        A[index] = x;  
    }  
}
```

50. Column-major `Get()` version

```
int Get(int i, int j) {  
    if (i >= j) {  
        int index = (n * (j - 1)) - ((j - 2) * (j - 1) / 2) + (i - j);  
        return A[index];  
    }  
}
```

```
    }  
    return 0;  
}
```

51. Column-major `Display()` version

```
void Display() {  
    for (int i = 1; i <= n; i++) {  
        for (int j = 1; j <= n; j++) {  
            if (i >= j) {  
                int index = (n * (j - 1)) - ((j - 2) * (j - 1) / 2) + (i - j);  
                cout << A[index] << " ";  
            } else {  
                cout << "0 ";  
            }  
        }  
        cout << endl;  
    }  
}
```

52. Why OOP makes the formula switch safer

OOP means object-oriented programming.

In Lesson 8, the formula is hidden inside the class methods.

Outside code only calls:

```
lm.Set(i, j, x);  
lm.Get(i, j);  
lm.Display();
```

The main program does not need to know whether the class uses row-major or column-major storage.

Only the class methods need to know that.

This makes the formula switch safer.

In simple words:

The user sees the same matrix.
The main program stays almost the same.
Only the internal formula changes.

This is an important advantage of using classes.

53. Stored array vs visible matrix

The visible matrix is what the user sees.

Example:

```
1 0 0 0
2 3 0 0
4 5 6 0
7 8 9 10
```

The stored array is what the computer actually stores.

For row-major storage:

```
A = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

The zeros are not stored.

They are printed or returned when needed.

This is the main memory-saving idea.

54. Beginner mental model

Think of the full matrix as what the user sees.

Think of the 1D array as what the computer stores.

For a lower triangular matrix:

The computer stores only values on and below the diagonal.
The computer does not store values above the diagonal.
The values above the diagonal are automatically treated as zero.

The class is like a protective box.

Inside the box:

A stores the useful values.
n stores the matrix size.
Set(), Get(), and Display() control access to the matrix.

Outside code does not directly touch `A`.

Outside code uses methods.

55. Complexity of Lesson 8

Object creation

The constructor allocates:

$N(N + 1) / 2$ values

Time complexity:

$O(1)$

Space complexity:

$O(N^2)$, optimized to about half of full matrix storage

It is still $O(N^2)$ because:

$N(N + 1) / 2$

still grows like N^2 as N becomes large.

Set() complexity

The **Set()** method does:

1. One condition check.
2. One formula calculation if needed.
3. One array assignment if needed.

So:

$$\text{Set()} = O(1)$$

Get() complexity

The **Get()** method does:

1. One condition check.
2. One formula calculation if needed.
3. One array lookup if needed.

So:

$$\text{Get()} = O(1)$$

Input reading complexity

The program reads all visible positions.

For an $N \times N$ matrix, that means:

$$N^2 \text{ inputs}$$

So:

$$\text{Input reading} = O(N^2)$$

Display() complexity

The display method prints every visible matrix position.

For an $N \times N$ matrix, that means:

N^2 positions

So:

$\text{Display()} = O(N^2)$

56. Complexity table

Operation	Time complexity	Space complexity	Reason
Object creation	$O(1)$	$O(N^2)$ optimized	Allocates $N(N + 1) / 2$ values.
Set()	$O(1)$	—	One condition and one formula.
Get()	$O(1)$	—	One condition and one possible array lookup.
Input reading	$O(N^2)$	—	Reads all visible matrix positions.
Display()	$O(N^2)$	—	Prints the full square matrix.

Important note:

Lower triangular storage is not $O(N)$.
It is $O(N^2)$, but it stores about half as many values as a full matrix.

57. Common beginner mistakes

Mistake 1: Using $i == j$

Wrong:

```
if (i == j)
```

Correct:

```
if (i >= j)
```

Why?

Because `i == j` is for diagonal matrices.

Lower triangular matrices store the diagonal and everything below it.

Mistake 2: Allocating only `n` cells

Wrong:

```
A = new int[n];
```

Correct:

```
A = new int[n * (n + 1) / 2]{};
```

Why?

A lower triangular matrix stores more than just the diagonal.

It needs:

```
n(n + 1) / 2 cells
```

Mistake 3: Allocating `n * n` cells

Wrong:

```
A = new int[n * n];
```

Correct:

```
A = new int[n * (n + 1) / 2]{};
```

Why?

`n * n` stores the whole matrix.

That wastes memory.

The purpose of special matrix storage is to avoid storing unnecessary zeros.

Mistake 4: Forgetting `delete[]`

Wrong:

```
~LowerTri() {  
    // no delete  
}
```

Correct:

```
~LowerTri() {  
    delete[] A;  
}
```

Why?

The class uses dynamic memory.

Dynamic memory must be released.

Mistake 5: Using `delete` instead of `delete[]`

Wrong:

```
delete A;
```

Correct:

```
delete[] A;
```

Why?

A points to an array.

Arrays created with `new[]` must be released using `delete[]`.

Mistake 6: Returning `-1` from `Get()`

Wrong:

```
return -1;
```

Correct:

```
return 0;
```

Why?

The actual matrix value above the diagonal is zero.

So `Get()` should return `0`.

Mistake 7: Changing the condition when switching to column-major

Wrong idea:

If storage changes, the matrix condition must change too.

Correct idea:

The matrix shape stays lower triangular.
The condition stays $i \geq j$.
Only the index formula changes.

58. Lesson 8 formula sheet

Lower triangular condition

$$i \geq j$$

This means the position is stored.

Structural zero condition

$$i < j$$

This means the position is above the diagonal and should be treated as zero.

Number of stored values

$$n * (n + 1) / 2$$

Number of structural zeros

$$n * (n - 1) / 2$$

Row-major index formula

$$\text{index} = i * (i - 1) / 2 + (j - 1)$$

Column-major index formula

$$\text{index} = n * (j - 1) - ((j - 2) * (j - 1) / 2) + (i - j)$$

59. Lesson 8 code summary

Class data

```
private:  
    int *A;  
    int n;
```


Constructor allocation

```
A = new int[n * (n + 1) / 2]{};
```

Destructor

```
delete[] A;
```

Set rule

```
if (i >= j) {  
    A[index] = x;  
}
```

Get rule

```
if (i >= j) {  
    return A[index];  
}  
return 0;
```

Display rule

```
if (i >= j) {  
    cout << A[index] << " ";  
} else {  
    cout << "0 ";  
}
```

60. Final summary

Lesson 8 converts the lower triangular matrix implementation from C into C++.

The main matrix rule stays:

```
i >= j
```

This means:

Store values on and below the main diagonal.
Treat values above the diagonal as zero.

The class stores:

```
int *A;  
int n;
```

The constructor allocates:

$$n * (n + 1) / 2$$

The destructor releases memory using:

```
delete[] A;
```

The row-major formula is:

$$\text{index} = i * (i - 1) / 2 + (j - 1)$$

The column-major formula is:

$$\text{index} = n * (j - 1) - ((j - 2) * (j - 1) / 2) + (i - j)$$

The most important design idea is:

The class hides the storage formula.
The main program uses Set(), Get(), and Display().
The formula can change internally without changing how the object is used.

That is why Lesson 8 is important.

It shows how special matrix storage and object-oriented programming work together.